2 L'Application

tester les deux procédures sur les vecteur colonnes des matrices suivantes

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & -1 & 2 \\ 1 & 1 & 2 \end{pmatrix} \qquad B = \begin{pmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{pmatrix} \quad \varepsilon = 10^{-10}$$

```
main.m
    e=10^(-10);
    A=(1,2,3;2,-1,2;1,1,2)
    A1=Schmidt_orthogonalization_Original(A)
    A2=Schmidt_orthogonalization_Modified(A)
    eA=Erreur_moyenne(A1,A2)
    B=(1,1,1;e,0,0;0,e,0;0,0,e)
    B1=Schmidt_orthogonalization_Original(B)
    B2=Schmidt_orthogonalization_Modified(B)
    eA=Erreur_moyenne(B1,B2)
end
```

les résultat obtenus:

A1 : le résultat de la version originale

```
A1 =

    0.4082    0.7591    0.5071
    0.8165   -0.5521    0.1690
    0.4082    0.3450   -0.8452
```

A2 : le résultat de la version modifiée

```
A2 =

    0.4082    0.7591    0.5071
    0.8165   -0.5521    0.1690
    0.4082    0.3450   -0.8452
```

B1 : le résultat de la version originale

```
B1 =

    1.0000         0         0
    0.0000   -0.7071   -0.7071
         0    0.7071         0
         0         0    0.7071
```

B2 : le résultat de la version modifiée

```
B2 =

    1.0000         0         0
    0.0000   -0.7071   -0.4082
         0    0.7071   -0.4082
         0         0    0.8165
```

Comparer et interpréter les résultats obtenus par les deux algorithmes

On peut voir que les résultats obtenus par les deux algorithmes parfois sont différents. Pour une recherche approfondie, nous créons une matrice E (100 * 100) égale à X1 - X2 et calculons sa norme Frobenius (racine carrée de la somme carrée de tous les termes de matrice), que nous définissons comme le nombre d'erreurs entre les deux versions orthogonalisées de Schmitt.

$$\|X\|_F \stackrel{\text{def}}{=} \sqrt{\sum_i \sum_j X_{i,j}^2}$$

la norme Frobenius

On obtient deux nombres d'erreurs:

eA: le nombre d'erreurs de A

eB: le nombre d'erreurs de B

```
eA =

   5.9242e-31
```

```
eB =

    0.2679
```

On peut voir que le nombre d'erreurs de A est zéro mais celui de B est grand.Pourquoi les résultats obtenus par les deux algorithmes parfois sont différents? Et pourquoi les résultats obtenus par les deux algorithmes parfois ne sont pas différents?

3 Les Explication de l'erreur

La version originale de l'orthogonalisation est comparable à la construction avec des blocs. On prend le premier vecteur comme référence et calcule le second vecteur perpendiculaire à cette référence. Puis on prend les deux vecteurs comme référence et calcule le troisième vecteur, et ainsi de suite.Le calcule du nième vecteur nécessite le résultat nth-1. Par conséquent, l'erreur d'un vecteur affectera la précision de chaque vecteur après celui-ci.

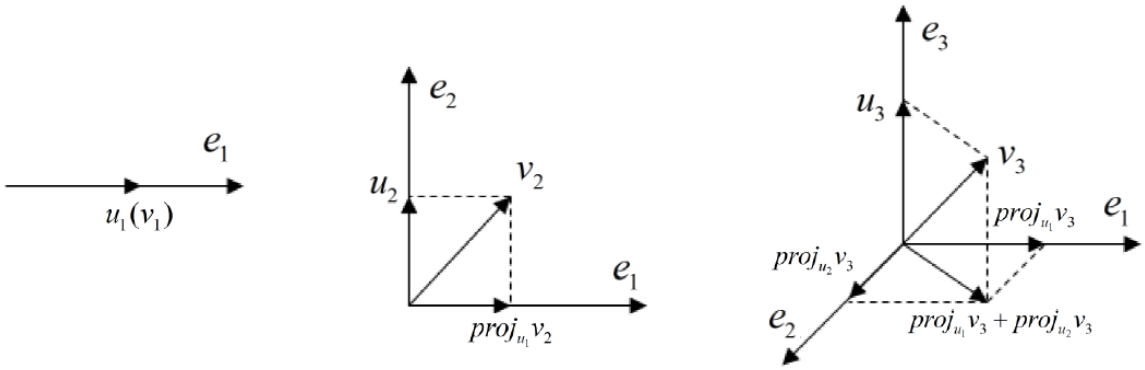


Figure 1 – Illustration de l'orthogonalisation de Schmidt lorsque k=1, 2, 3

De plus, la façon dont les ordinateurs sont calculés peut produire des résidus, en particulier lors de la soustraction de deux nombres proches l'un de l'autre. Les erreurs sont amplifiées comme une boule de neige au fur et à mesure que le calcul se poursuit. La nouvelle méthode calcule chaque composante de uk séparément, et bien qu'elles puissent également générer des résidus, les erreurs ne sont pas amplifiées.

4 Le Lien entre l'erreur et le nombre de conditions

Pourquoi le nombre d'erreurs de A est zéro mais celui de B est grand. Quels sont les différences entre A et B.

On calcule les deux nombres conditionnelles de A et B:
le nombre conditionnelle de A = 53.9413
le nombre conditionnelle de B = 1.7321e+10

Donc nous supposons que plus le nombre de conditions est élevé, peut-être plus l'erreur entre les deux versions est élevé. On fait un programme pour le vérifier.

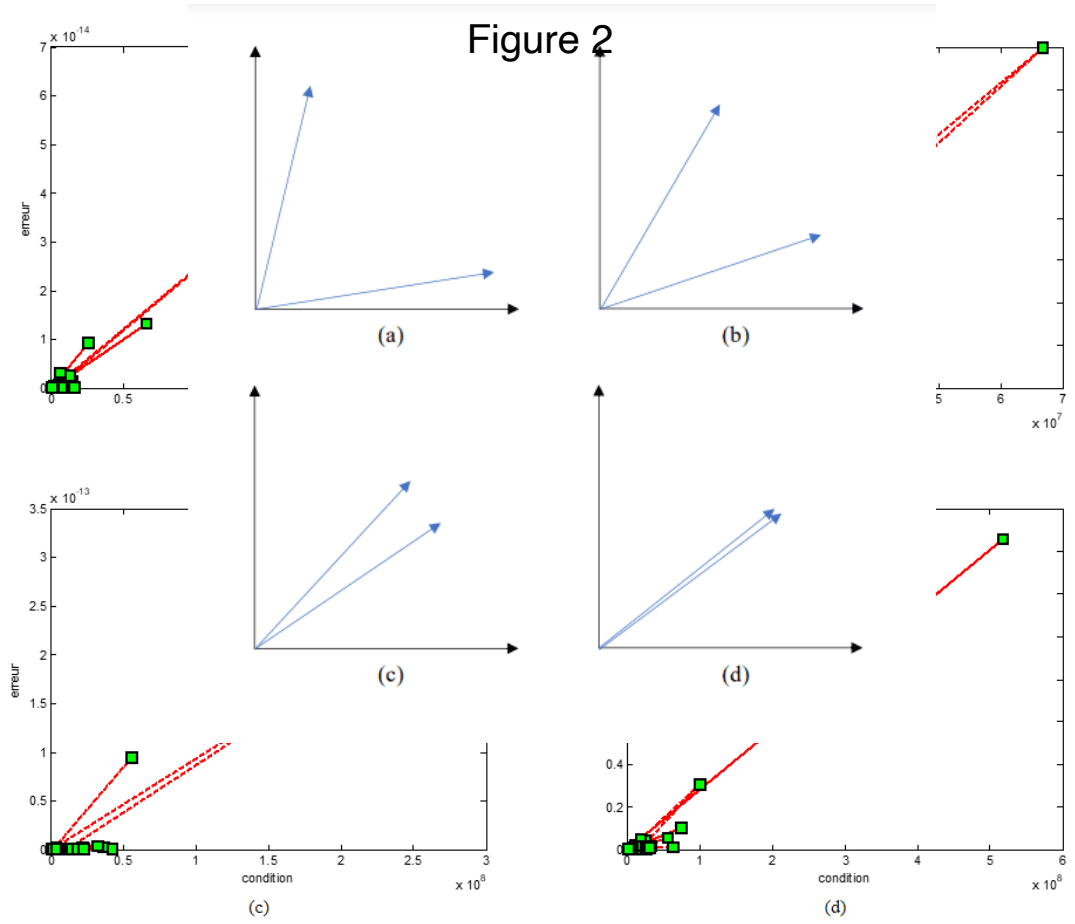
```
main.m
c=zeros(20,1);
s=zeros(20,1);
for i=1:20
    a=rand(20);
    while(cond(a)<1000000000)
       a=rand(20);
    end
    x1=Schmidt_orthogonalization_Original(a);
```

```
    x2=Schmidt_orthogonalization_Modified(a);
    s(i)=Erreur_moyenne(x1,x2);
    c(i)=cond(a);
end
plot(c,s,'--rs','LineWidth',2,...
                'MarkerEdgeColor','k',...
                'MarkerFaceColor','g',...
                'MarkerSize',10)
xlabel('condition');
ylabel('erreur');
```

Le but de cet algorithme est de montrer la relation entre l'erreur entre les deux versions d'orthogonalisation de Schmidt et le nombre de conditions dans un graphique linéaire. Il crée d'abord au hasard 100 matrices(100 * 100) dont le nombre de conditions est supérieur à 1 000 000. Un million, C'est nécessaire, nous avons besoin d'un plus grand nombre d'erreurs pour que le diagramme linéaire semble meilleur. Mais s'il est plus élevé, il faudra peut-être une journée entière pour produire un graphique. Ensuite, à travers les deux fonctions de chaque version, nous obtenons une matrice X1 (100 * 100) de la version originale (Schmidt_orthogonalization_original.m) et une autre matrice x2 (100 * 100) de la version modifiée (Schmidt_orthogonalization_modified.m). Ensuite, nous créons une matrice E (100 * 100) égale à X1 - X2 et calculons sa norme Frobenius (racine carrée de la somme carrée de tous les termes de matrice), que nous définissons comme le nombre d'erreurs entre les deux versions orthogonalisées de Schmitt. Avec 100 matrices, nous avons maintenant 100 nombres conditionnels et leurs erreurs associées. Enfin, l'utilisez pour créer le graphique linéaire.L'axe horizontal est le nombre conditionnel, l'axe vertical est le nombre d'erreurs entre les deux versions orthogonalisées de Schmitt.

Nous avons exécuté le programme 4 fois, avec 4 diagrammes linéaires. (Figure 1) nous pouvons constater qu'il semble y avoir une certaine relation entre l'erreur entre les deux versions de l'orthogonalisation de Schmidt et le nombre de conditions. Bien que l'erreur soit très faible, nous pouvons trouver que, plus la valeur nominale de la condition, plus les deux versions de la valeur nominale.
En prenant la matrice (2 * 2) par exemple, nous avons maintenant deux vecteurs, colonnes A1 et A2. On dessine en coordonnées cartésiennes. Si cette matrice est de rang complet, A1 et A2 doivent être libres, ce qui est nécessaire pour effectuer

Figure 2



l'orthogonalisation Schmidt. En d'autres termes, les deux vecteurs ne peuvent pas être parallèles. Mais si nous tournons lentement l'un de ces vecteurs de sorte qu'ils soient presque parallèles. (Figure 2) l'ordinateur peut les considérer comme deux vecteurs parallèles, même s'ils ne sont pas strictement parallèles. Cela aggrave les problèmes causés par les résidus et conduit à une plus grande erreur entre les deux versions de l'orthogonalisation de Schmidt.

Note:

erreur_moyenne.m

```
function s=Erreur_moyenne(a,b)
    d=a-b;
    s=0;
    [m,n] = size(a);
```

```
    for i=1:m
        for j=1:n
            s=s+d(i,j)^2;
        end
    end
end
```

Schmidt_orthogonalization_Modified.m

```
function b=Schmidt_orthogonalization_Modified(a)
    [m,n] = size(a);
    b=zeros(m,n);
    b(:,1)=a(:,1);
    for i=2:n
        for j=1:i-1
            b(:,i)=a(:,i)-dot(a(:,i),b(:,j))/dot(b(:,j),b(:,j))*b(:,j);
            a(:,i)=b(:,i);
        end
    end
    for k=1:n
    b(:,k)=b(:,k)/norm(b(:,k));
end
```

Schmidt_orthogonalization_Original.m

```
function b=Schmidt_orthogonalization_Original(a)
    [m,n] = size(a);
    b=zeros(m,n);
    b(:,1)=a(:,1);
    for i=2:n
        for j=1:i-1
            b(:,i)=b(:,i)-dot(a(:,i),b(:,j))/dot(b(:,j),b(:,j))*b(:,j);
```

```
        end
        b(:,i)=b(:,i)+a(:,i);
    end
    for k=1:n
    b(:,k)=b(:,k)/norm(b(:,k));
end
```

This algorithm is designed to show the link between the error between two versions of Schmidt orthogonalization and the condition number in a line chart. It first create randomly 100 matrix(100*100) whose condition number are bigger than 1000000. Why doing so? Because these two versions are both correct in terms of mathematics but differ in the steps of calculation. The larger its condition number is, the closer its determinant will be to zero, and more likely the error between two versions will increase. But 1000000 is enough, if higher, that could take a whole day to form the chart. Then through those two functions of each version, we have a matrix x1(100*100) from the original version(Schmidt_orthogonalization_Original.m) and another matrix x2(100*100) from the modified version (Schmidt_orthogonalization_Modified.m). Later we create a matrix e(100*100) that equals to x1-x2, and calculate the Frobenius norm of it (the square root of the sums of the squares of all the matrix entries), we define this calculated number as the error number between two versions of Schmidt orthogonalization.With 100 matrix, we now have 100 condition numbers and their related error number. So in the end, a chart is formed using it.

We run the program for 4 times and have 4 line charts.(image 1) From which we can discover that there seems to exist some connections between the error between two versions of Schmidt orthogonalization and the condition number. Although the error is a very very small number, we can find that plus le nombre de conditions est élevé, plus l'erreur entre les deux versions est élevé. Why is that?

Take a matrix(2*2) as an exemple, now we have two vectors colonnes A1 and A2. We draw it in cartesian coordinates. If this matrix is full rank , A1 and A2 should be libre and that’s a condition necessary to perform the Schmidt orthogonalization. That means, these two vectors could not be parallel. Well, but if we slowly turn one of the vectors to make them nearly parallel.(image 2) It can be considered by the computer as two parallel vectors even that they are not strictly parallel. This enhance the problem caused by the residual and result in a larger error between two versions of Schmidt orthogonalization.